

A decorative element consisting of three vertical lines of varying heights and colors (yellow, orange, and blue) on the left side of the slide.

2. Pruebas dinámicas de *software*

Depuración

Miguel Ángel Latre
Javier Nogueras Iso

Índice

Depuración

❖ Técnicas

- Fuerza bruta
- Inducción
- Deducción
- Traza inversa (*backtracking*)
- *Testing*

❖ Principios

Depuración

- Es el proceso que sigue a la ejecución de una prueba exitosa (es decir, la prueba que detecta un fallo o un error).
- La depuración es un proceso de dos fases:
 - ❖ Determinar la naturaleza exacta y localización en el código del defecto
 - ❖ Corregir el defecto

Depuración

□ Técnicas

❖ Fuerza bruta

- Volcado de memoria (*core dump*)
- Chivatos
- Depuradores

❖ Inducción

❖ Deducción

❖ Traza inversa (*backtracking*)

❖ *Testing*

Volcado de memoria (*core dump*)

- ❑ Es el estado completo de un programa en ejecución en un determinado momento.
 - ❖ Habitualmente, el momento en el que se produce un error que fuerza a su terminación
- ❑ Compuesto por:
 - ❖ El contenido completo de la memoria utilizada por el programa
 - ❖ Contenido de los registros, contador del programa, *flags*, ...
- ❑ Se pueden utilizar para depuración.

Chivatos

- ❑ Instrucciones de escritura en pantalla para informar sobre el estado del programa
- ❑ Desventajas:
 - ❖ Requieren modificar el código fuente
 - Pueden enmascarar el defecto buscado
 - Pueden provocar otros errores
 - ❖ Requieren una consola o *log* en la que escribir
 - ❖ Puede producir gran cantidad de información
 - Posiblemente no estructurada

Chivatos

```
Entrando en analizarCompras("Cliente12634.dat", 2008, 2012)
Entrando en calcular("Cliente12634.dat", 2008, 2012, ventas)
Iteración 1: Venta=[11, 900000, 3.220000, 20121114]
Iteración 2: Venta=[73, 630000, 1.000000, 20121112]
Iteración 3: Venta=[92, 280000, 13.200000, 20100914]
...
Iteración 105: Venta=[25, 580000, 0.170000, 20100305]
Saliendo de calcular()
Fin de analizarCompras()
```

Chivatos

estoy aquí

Estoy aquí 2

123456789fuera de rango!

1011121314151617fuera de rango!

18192021fuera de rango!

...

90919293fuera de rango!

949596979899100101102fuera de rango!

103104105106Fin

fin

Depuradores

- ❑ No requieren modificar el código fuente
- ❑ No requieren una consola o *log* en la que escribir
- ❑ Pueden mostrar gran cantidad de información
 - ❖ Que la persona que depura suele analizar sobre la marcha

Índice

Depuración

❖ Técnicas

- Fuerza bruta
- **Inducción**
- Deducción
- Traza inversa (*backtracking*)
- *Testing*

❖ Principios

Inducción

❏ Pasos

- ❖ Analizar los errores detectados en las pruebas
- ❖ Formular una hipótesis acerca del defecto
- ❖ Probar la hipótesis
 - Analizando el código fuente
 - Con el depurador
 - Añadiendo más pruebas
- ❖ Corregir el defecto

Ejemplo

❏ Problema «Fizz Buzz»

- ❖ Programa que escribe en la pantalla los números del 1 al 100, con las siguientes excepciones:
 - Se debe escribir «Fizz», en lugar del número correspondiente, si el número es divisible por 3 o contiene el dígito 3.
 - Se debe escribir «Buzz», en lugar del número correspondiente, si el número es divisible por 5 o contiene el dígito 5.
 - Se debe escribir «FizzBuzz», en lugar del número correspondiente, si el número es divisible por 3 y por 5 o contiene los dígitos 3 y 5.

Ejemplo

```
1  
2  
Fizz  
4  
Buzz  
Fizz  
7  
8  
Fizz  
Buzz  
11  
Fizz  
Fizz  
14  
FizzBuzz
```

Ejemplo

- ❑ Supongamos que en el programa FizzBuzz hay algún defecto.
- ❑ El resultado de las pruebas es el siguiente:
 - ❖ testThat01Returns01() → OK
 - ❖ testThat02Returns02() → OK
 - ❖ testThat03ReturnsFizz() → OK
 - ❖ testThat05ReturnsBuzz() → OK
 - ❖ testThat06ReturnsFizz() → OK
 - ❖ testThat10ReturnsBuzz() → OK
 - ❖ testThat13ReturnsFizz() → OK
 - ❖ testThat15ReturnsFizzBuzz() → expected:<FizzBuzz> but was:<Buzz>
 - ❖ testThat30ReturnsFizzBuzz() → expected:<FizzBuzz> but was:<Buzz>
 - ❖ testThat51ReturnsFizzBuzz() → expected:<FizzBuzz> but was:<Buzz>
 - ❖ testThat53ReturnsFizzBuzz() → expected:<FizzBuzz> but was:<Buzz>

Solución

```
...  
if (isFizzNumber(number)) {  
    result = FIZZ_STRING;  
}
```

```
if (isBuzzNumber(number)) {  
    result = BUZZ_STRING;  
}
```

```
...
```

```
...  
if (isBuzzNumber(number)) {  
    result += BUZZ_STRING;  
}
```

```
...
```


Ejemplo

- ❑ En otra versión del programa FizzBuzz hay otro defecto.
- ❑ El resultado de las pruebas es el siguiente:
 - ❖ `testThat01Returns01()` → OK
 - ❖ `testThat02Returns02()` → OK
 - ❖ `testThat03ReturnsFizz()` → OK
 - ❖ `testThat05ReturnsBuzz()` → OK
 - ❖ `testThat06ReturnsFizz()` → expected:<[Fizz]> but was:<[6]>
 - ❖ `testThat10ReturnsBuzz()` → OK
 - ❖ `testThat13ReturnsFizz()` → expected:<[Fizz]> but was:<[13]>
 - ❖ `testThat15ReturnsFizzBuzz()` → expected:<FizzBuzz> but was:<Buzz>
 - ❖ `testThat30ReturnsFizzBuzz()` → OK
 - ❖ `testThat51ReturnsFizzBuzz()` → expected:<FizzBuzz> but was:<Buzz>
 - ❖ `testThat53ReturnsFizzBuzz()` → expected:<FizzBuzz> but was:<Buzz>

Solución

```
private boolean isFizzNumber(int number) {  
    return isMultipleOf(number, FIZZ_NUMBER)  
        && containsNumber(number, FIZZ_NUMBER);  
}
```

```
private boolean isFizzNumber(int number) {  
    return isMultipleOf(number, FIZZ_NUMBER)  
        || containsNumber(number, FIZZ_NUMBER);  
}
```

Deducción

❏ Pasos

- ❖ Enumerar las posibles causas o hipótesis
- ❖ Utilizar datos de las pruebas para eliminar algunas de las posibles causas
- ❖ Refinar las hipótesis restantes
- ❖ Probar las hipótesis
- ❖ Corregir el defecto

Ejemplo

- ❑ En el programa FizzBuzz se ha introducido otro defecto.
- ❑ Posibles causas:
 - ❖ No se determina correctamente cuando un número es múltiplo de 3.
 - ❖ No se determina correctamente cuando un número contiene el dígito 3.
 - ❖ No se determina correctamente cuando un número es múltiplo de 5.
 - ❖ No se determina correctamente cuando un número contiene el dígito 5.
 - ❖ No se determina correctamente cuando un número es múltiplo de 3 y 5.
 - ❖ No se determina correctamente cuando un número contiene el dígito 3 y el 5.
 - ❖ No se devuelve “Fizz” cuando el número es múltiplo de 3 o contiene el dígito 3.
 - ❖ No se devuelve “Buzz” cuando el número es múltiplo de 5 o contiene el dígito 5.
 - ❖ No se devuelve “FizzBuzz” cuando el número es múltiplo de 3 y 5 o contiene el dígito 3 y el 5.

Ejemplo

□ El resultado de las pruebas es el siguiente:

- ❖ `testThat01ReturnsOne()` → OK
- ❖ `testThat02ReturnsTwo()` → OK
- ❖ `testThat03ReturnsFizz()` → OK
- ❖ `testThat05ReturnsBuzz()` → OK
- ❖ `testThat06ReturnsFizz()` → OK
- ❖ `testThat10ReturnsBuzz()` → OK
- ❖ `testThat13ReturnsFizz()` → Expected: “Fizz”, but was: “13”
- ❖ `testThat15ReturnsFizzBuzz()` → OK
- ❖ `testThat30ReturnsFizzBuzz()` → OK
- ❖ `testThat51ReturnsFizzBuzz()` → Expected: “FizzBuzz”, but was: “Fizz”
- ❖ `testThat53ReturnsFizzBuzz()` → Expected: “FizzBuzz”, but was: “53”

Ejemplo

Posibles causas

- ☐ ~~¿Múltiplo de 3?~~
- ☐ ¿Contiene el 3?
- ☐ ~~¿Múltiplo de 5?~~
- ☐ ¿Contiene el 5?
- ☐ ~~¿Múltiplo de 3 y 5?~~
- ☐ ¿Contiene el 3 y el 5?
- ☐ ¿Devuelve “Fizz” con múltiplos de 3 o cuando contienen el dígito 3?
- ☐ ¿Devuelve “Buzz” con múltiplos de 5 o cuando contienen el dígito 5?
- ☐ ¿Devuelve “FizzBuzz” con múltiplos de 3 o 5 o cuando contienen el dígito 3 o 5?

Resultados de los tests

- ☐ Números *normales*:
 - ❖ 1, 2 → OK
- ☐ Números *fizz*:
 - ❖ 3, 6 → OK
 - ❖ 13 → Expected: “Fizz”, but was: “13”
- ☐ Números *buzz*:
 - ❖ 5, 10 → OK
- ☐ Números *fizzbuzz*:
 - ❖ 15, 30 → OK
 - ❖ 51 → Expected: “FizzBuzz”, but was: “Fizz”
 - ❖ 53 → Expected: “FizzBuzz”, but was: “53”

Ejemplo

❑ Causas más básicas:

- ❖ No se determina correctamente cuando un número contiene el dígito 3.
- ❖ No se determina correctamente cuando un número contiene el dígito 5.

❑ Hipótesis:

- ❖ No se determina correctamente cuando un número contiene un determinado dígito.

Solución

```
public boolean containsDigit(int digit) {  
    String str = String.valueOf(number);  
    return str.contains(" " + digit);  
}
```

```
public boolean containsDigit(int digit) {  
    String str = String.valueOf(number);  
    return str.contains(" " + digit);  
}
```

```
public boolean containsDigit(int digit) {  
    String str = String.valueOf(number);  
    return str.contains(String.valueOf(digit));  
}
```


Traza inversa (*backtraking*)

□ Pasos

- ❖ Partir de la instrucción en la que se produce un resultado erróneo
- ❖ Analizar las instrucciones anteriores, en orden inverso de ejecución, para localizar dónde está el defecto
 - Primera instrucción en la que el estado real del programa difiere del estado esperado

□ Aplicación directa de los conceptos de verificación formal:

- ❖ Precondición más débil, pero con datos concretos resultantes de una ejecución del programa

Testing

- Profundizar en los casos de prueba que detectan errores, añadiendo nuevos casos:
 - ❖ Aplicación de técnicas con mayor grado de profundidad
 - Caminos con profundidad
 - Pares (o tríos, ...) de parámetros
 - Valores límite ± 1 , ± 2 , ...

Depuración

□ Principios

- ❖ Evitar la experimentación:

- Ejemplo: cambiar algo en alguna instrucción donde se sospecha que está el error para “ver qué pasa”.

- ❖ Donde hay un defecto, es probable que haya más.

- ❖ **Corrección del defecto, no de su síntoma**

- ❖ La corrección de un defecto puede introducir otro:

- En programas grandes, 1 de cada 6 defectos corresponde con la corrección de un defecto anterior.



PRUEBAS DINÁMICAS DE SOFTWARE

Depuración: Problema de la práctica 5

Problema

❏ *The Gilded Rose Kata*

❖ <https://iamnotmyself.com/refactor-this-the-gilded-rose-kata/>

❖ En el aula

- Lectura del enunciado
- Diseño de pruebas de caja negra

❖ En el laboratorio (práctica 5.ª)

- Descarga de código y pruebas
- Análisis de las pruebas existentes y las diseñadas
- Medición de la cobertura de las pruebas
- Análisis de los resultados de las pruebas
- Depuración

The Gilded Rose kata

- Hi and welcome to team Gilded Rose. As you know, we are a small inn with a prime location in a prominent city ran by a friendly innkeeper named Allison. We also buy and sell only the finest goods. Unfortunately, our goods are constantly degrading in quality as they approach their sell by date. We have a system in place that updates our inventory for us. It was developed by a no-nonsense type named Leeroy, who has moved on to new adventures. Your task is to add the new feature to our system so that we can begin selling a new category of items.

The Gilded Rose kata

□ First an introduction to our system:

- ❖ All items have a `SellIn` value which denotes the number of days we have to sell the item.
- ❖ All items have a `Quality` value which denotes how valuable the item is.
- ❖ At the end of each day our system lowers both values for every item.

The Gilded Rose kata

- ❑ Pretty simple, right? Well this is where it gets interesting:
 - ❖ Once the sell by date has passed, Quality degrades twice as fast.
 - ❖ The Quality of an item is never negative.
 - ❖ "Aged Brie" actually increases in Quality the older it gets.
 - ❖ The Quality of an item is never more than 50.
 - ❖ "Sulfuras", being a legendary item, never has to be sold or decreases in Quality.
 - ❖ "Backstage passes", like aged brie, increases in Quality as it's SellIn value approaches; Quality increases by 2 when there are 10 days or less and by 3 when there are 5 days or less but Quality drops to 0 after the concert.

The Gilded Rose kata

- ❑ We have recently signed a supplier of conjured items. This requires an update to our system:
 - ❖ "Conjured" items degrade in Quality twice as fast as normal items
- ❑ Feel free to make any changes to the UpdateQuality method and add any new code as long as everything still works correctly. However, do not alter the Item class or Items property as those belong to the goblin in the corner who will insta-rage and one-shot you as he doesn't believe in shared code ownership (you can make the UpdateQuality method and Items property static if you like, we'll cover for you).
- ❑ Just for clarification, an item can never have its Quality increase above 50, however "Sulfuras" is a legendary item and as such its Quality is 80 and it never alters.

The Gilded Rose kata

Item
-name : String -quality : int -sellIn : int
+Item() +getName() +setName() +getQuality() +setQuality() +getSellIn() +setSellIn()

The Gilded Rose kata

□ Diseño de casos de prueba

- ❖ Usaremos la técnica de pares de parámetros

- ❖ Suponed en otra clase un método

public static void update(Item item)

Clases de equivalencia

Parámetro	Válidas	No válidas
item.name		

Clases de equivalencia

Parámetro	Válidas	No válidas
item.name	"Aged Brie"	
	"Sulfuras"	
	"Backstage passes"	

Clases de equivalencia

Parámetro	Válidas	No válidas
item.name	"Aged Brie"	
	"Sulfuras"	
	"Backstage passes"	
	Otros	

Clases de equivalencia

Parámetro	Válidas	No válidas
item.name	"Aged Brie"	"" null
	"Sulfuras"	
	"Backstage passes"	
	Otros	

Clases de equivalencia

Parámetro	Válidas	No válidas
item.name	"Aged Brie" "Sulfuras" "Backstage passes" Otros	"" null
item.sellIn		

Clases de equivalencia

Parámetro	Válidas	No válidas
item.name	"Aged Brie" "Sulfuras" "Backstage passes" Otros	"" null
item.sellIn	≥ 0 < 0	

Clases de equivalencia

Parámetro	Válidas	No válidas
item.name	"Aged Brie" "Sulfuras" "Backstage passes" Otros	"" null
item.sellIn	> 10 6..10 1..5 0 < 0	

Clases de equivalencia

Parámetro	Válidas	No válidas
item.name	"Aged Brie" "Sulfuras" "Backstage passes" Otros	"" null
item.sellIn	> 10 6..10 1..5 0 < 0	
	∞	$< \infty$

Clases de equivalencia

Parámetro	Válidas	No válidas
item.name	"Aged Brie" "Sulfuras" "Backstage passes" Otros	"" null
item.sellIn	> 10 6..10 1..5 0 < 0	
	∞	$< \infty$
item.quality		

Clases de equivalencia

Parámetro	Válidas	No válidas
item.name	"Aged Brie" "Sulfuras" "Backstage passes" Otros	"" null
item.sellIn	> 10 6..10 1..5 0 < 0	
	∞	$< \infty$
item.quality	0..50	

Clases de equivalencia

Parámetro	Válidas	No válidas
item.name	"Aged Brie" "Sulfuras" "Backstage passes" Otros	"" null
item.sellIn	> 10 6..10 1..5 0 < 0	
	∞	$< \infty$
item.quality	0..50	> 50 < 0

Clases de equivalencia

Parámetro	Válidas	No válidas
item.name	"Aged Brie" "Sulfuras" "Backstage passes" Otros	"" null
item.sellIn	> 10 6..10 1..5 0 < 0	
	∞	$< \infty$
item.quality	0..50	> 50 < 0
	80	$\neq 80$

Clases de equivalencia

Parámetro	Válidas	No válidas
item.name	1 "Aged Brie"	14 ""
	2 "Sulfuras"	15 null
	3 "Backstage passes"	
	4 Otros	
item.sellIn	7 > 10	
	8 6..10	
	9 1..5	
	10 0	
	20 < 0	
	11 ∞	16 < ∞
item.quality	12 0..50	17 > 50
		18 < 0
	13 80	19 $\neq$ 80

Pruebas de pares.

Situaciones de prueba

□ Salen 44:

- ❖ 1-7, 1-8, 1-9, 1-10, 1-20, 1-11, 1-12, 1-13,
- ❖ 2-7, 2-8, 2-9, 2-10, 2-20, 2-11, 2-12, 2-13,
- ❖ 3-7, 3-8, 3-9, 3-10, 3-20, 3-11, 3-12, 3-13,
- ❖ 4-7, 4-8, 4-9, 4-10, 4-20, 4-11, 4-12, 4-13,
- ❖ 7-12, 7-13,
- ❖ 8-12, 8-13,
- ❖ 9-12, 9-13,
- ❖ 10-12, 10-13,
- ❖ 20-12, 20-13,
- ❖ 11-12, 11-13

Pruebas de pares.

Situaciones de prueba

❑ Salen 44:

- ❖ 1-7, 1-8, 1-9, 1-10, 1-20, 1-11, 1-12, 1-13, 2-7, 2-8, 2-9, 2-10, 2-20, 2-11, 2-12, 2-13, 3-7, 3-8, 3-9, 3-10, 3-20, 3-11, 3-12, 3-13, 4-7, 4-8, 4-9, 4-10, 4-20, 4-11, 4-12, 4-13, 7-12, 7-13, 8-12, 8-13, 9-12, 9-13, 10-12, 10-13, 20-12, 20-13, 11-12, 11-13

❑ Muchas son imposibles:

- ❖ Sulfuras (clase 2) solo puede ir con sellIn = ∞ (clase 11) y quality = 80 (clase 13)
- ❖ Por ello, la clase 2 es incompatible con las clases 7 a 10 y 20; la clase 11 es incompatible con las clases 1, 3, 4 y 12; la clase 13 es incompatible con las clases 1, 3, 4 y 7 a 10

❑ Muchas son demasiado redundantes:

- ❖ Las clases 7 a 10 son útiles para la clase 3 (Backstage passes). Para el resto, bastaría con distinguir entre sellIn ≥ 0 y sellIn < 0

Clases de equivalencia (sellIn adaptado)

Parámetro	Válidas	No válidas
item.name	1 "Aged Brie"	14 ""
	2 "Sulfuras"	15 null
	3 "Backstage passes"	
	4 Otros	
item.sellIn	5 > 0	
	6 ≤ 0	
	7 > 10	
	8 6..10	
	9 1..5	
	10 0	
item.quality	11 ∞	16 < ∞
	12 0..50	17 > 50
		18 < 0
	13 80	19 ≠ 80

□ En las situaciones de prueba se utilizarán las clases 5 y 6 con las clases 1 y 4

Clases de equivalencia (sellIn adaptado)

Parámetro	Válidas	No válidas
item.name	1 "Aged Brie"	14 ""
	2 "Sulfuras"	15 null
	3 "Backstage passes"	
	4 Otros	
item.sellIn	5 > 0	
	6 ≤ 0	
	7 > 10	
	8 6..10	
	9 1..5	
	10 0	
	11 ∞	16 < ∞
item.quality	12 0..50	17 > 50
		18 < 0
	13 80	19 ≠ 80

- En las situaciones de prueba se utilizarán las clases 5 y 6 con las clases 1 y 4
- Las clases 7 a 10 con la clase 3

Clases de equivalencia (sellIn adaptado)

Parámetro	Válidas	No válidas
item.name	1 "Aged Brie"	14 ""
	2 "Sulfuras"	15 null
	3 "Backstage passes"	
	4 Otros	
item.sellIn	5 > 0	
	6 ≤ 0	
	7 > 10	
	8 6..10	
	9 1..5	
	10 0	
	11 ∞	16 < ∞
item.quality	12 0..50	17 > 50
		18 < 0
	13 80	19 ≠ 80

- ❑ En las situaciones de prueba se utilizarán las clases 5 y 6 con las clases 1 y 4
- ❑ Las clases 7 a 10 con la clase 3
- ❑ La clase 11 con la clase 2

Clases de equivalencia y situaciones de prueba

Parámetro	Válidas	No válidas
item.name	1 "Aged Brie"	14 ""
	2 "Sulfuras"	15 null
	3 "Backstage passes"	
	4 Otros	
item.sellIn	5 > 0	
	6 ≤ 0	
	7 > 10	
	8 6..10	
	9 1..5	
	10 0	
	11 ∞	16 < ∞
item.quality	12 0..50	17 > 50
		18 < 0
	13 80	19 ≠ 80

Situaciones de prueba	
1	5
1	6
1	12
2	11
2	13
3	6
3	7
3	8
3	9
3	10
3	12
4	5
4	6
4	12
5	12
6	12
7	12
8	12
9	12
10	12
11	13

Casos de prueba

Prueba	name	sellIn	quality	Clases	Resultado		
					Pares	sellIn	quality
P1	"Aged Brie"	10	25	1 5 12	1-5, 1-12, 5-12	9	26

Casos de prueba

Prueba	name	sellIn	quality	Clases	Resultado		
					Pares	sellIn	quality
P1	"Aged Brie"	10	25	1 5 12	1-5, 1-12, 5-12	9	26
P2	"Aged Brie"	-8	25	1 6 12	1-6, 6-12	-9	27

Casos de prueba

Prueba	name	sellIn	quality	Clases	Resultado		
					Pares	sellIn	quality
P1	"Aged Brie"	10	25	1 5 12	1-5, 1-12, 5-12	9	26
P2	"Aged Brie"	-8	25	1 6 12	1-6, 6-12	-9	27
P3	"Sulfuras"	*	80	2 11 13	2-11, 2-13, 11-13	*	80

* Integer.MAX_VALUE

Casos de prueba

Prueba	name	sellIn	quality	Clases	Resultado		
					Pares	sellIn	quality
P1	"Aged Brie"	10	25	1 5 12	1-5, 1-12, 5-12	9	26
P2	"Aged Brie"	-8	25	1 6 12	1-6, 6-12	-9	27
P3	"Sulfuras"	*	80	2 11 13	2-11, 2-13, 11-13	*	80
P4	"Backstage passes"	11	25	3 7 12	3-7, 3-12, 7-12	10	26

* Integer.MAX_VALUE

Casos de prueba

Prueba	name	sellIn	quality	Clases	Resultado		
					Pares	sellIn	quality
P1	"Aged Brie"	10	25	1 5 12	1-5, 1-12, 5-12	9	26
P2	"Aged Brie"	-8	25	1 6 12	1-6, 6-12	-9	27
P3	"Sulfuras"	*	80	2 11 13	2-11, 2-13, 11-13	*	80
P4	"Backstage passes"	11	25	3 7 12	3-7, 3-12, 7-12	10	26
P5	"Backstage passes"	9	25	3 8 12	3-8, 8-12	8	27

* Integer.MAX_VALUE

Casos de prueba

Prueba	name	sellIn	quality	Clases	Resultado		
					Pares	sellIn	quality
P1	"Aged Brie"	10	25	1 5 12	1-5, 1-12, 5-12	9	26
P2	"Aged Brie"	-8	25	1 6 12	1-6, 6-12	-9	27
P3	"Sulfuras"	*	80	2 11 13	2-11, 2-13, 11-13	*	80
P4	"Backstage passes"	11	25	3 7 12	3-7, 3-12, 7-12	10	26
P5	"Backstage passes"	9	25	3 8 12	3-8, 8-12	8	27
P6	"Backstage passes"	4	25	3 9 12	3-9, 9-12	3	28

* Integer.MAX_VALUE

Casos de prueba

Prueba	name	sellIn	quality	Clases	Resultado		
					Pares	sellIn	quality
P1	"Aged Brie"	10	25	1 5 12	1-5, 1-12, 5-12	9	26
P2	"Aged Brie"	-8	25	1 6 12	1-6, 6-12	-9	27
P3	"Sulfuras"	*	80	2 11 13	2-11, 2-13, 11-13	*	80
P4	"Backstage passes"	11	25	3 7 12	3-7, 3-12, 7-12	10	26
P5	"Backstage passes"	9	25	3 8 12	3-8, 8-12	8	27
P6	"Backstage passes"	4	25	3 9 12	3-9, 9-12	3	28
P7	"Backstage passes"	0	25	3 10 12	3-10, 10-12	-1	0

* Integer.MAX_VALUE

Casos de prueba

Prueba	name	sellIn	quality	Clases	Resultado		
					Pares	sellIn	quality
P1	"Aged Brie"	10	25	1 5 12	1-5, 1-12, 5-12	9	26
P2	"Aged Brie"	-8	25	1 6 12	1-6, 6-12	-9	27
P3	"Sulfuras"	*	80	2 11 13	2-11, 2-13, 11-13	*	80
P4	"Backstage passes"	11	25	3 7 12	3-7, 3-12, 7-12	10	26
P5	"Backstage passes"	9	25	3 8 12	3-8, 8-12	8	27
P6	"Backstage passes"	4	25	3 9 12	3-9, 9-12	3	28
P7	"Backstage passes"	0	25	3 10 12	3-10, 10-12	-1	0
P8	"Backstage passes"	-8	0	3 6 12	3-6	-9	0

* Integer.MAX_VALUE

Casos de prueba

Prueba	name	sellIn	quality	Clases	Resultado		
					Pares	sellIn	quality
P1	"Aged Brie"	10	25	1 5 12	1-5, 1-12, 5-12	9	26
P2	"Aged Brie"	-8	25	1 6 12	1-6, 6-12	-9	27
P3	"Sulfuras"	*	80	2 11 13	2-11, 2-13, 11-13	*	80
P4	"Backstage passes"	11	25	3 7 12	3-7, 3-12, 7-12	10	26
P5	"Backstage passes"	9	25	3 8 12	3-8, 8-12	8	27
P6	"Backstage passes"	4	25	3 9 12	3-9, 9-12	3	28
P7	"Backstage passes"	0	25	3 10 12	3-10, 10-12	-1	0
P8	"Backstage passes"	-8	0	3 6 12	3-6	-9	0
P9	"Dexterity Vest"	10	20	4 5 12	4-5, 4-12	9	19

* Integer.MAX_VALUE

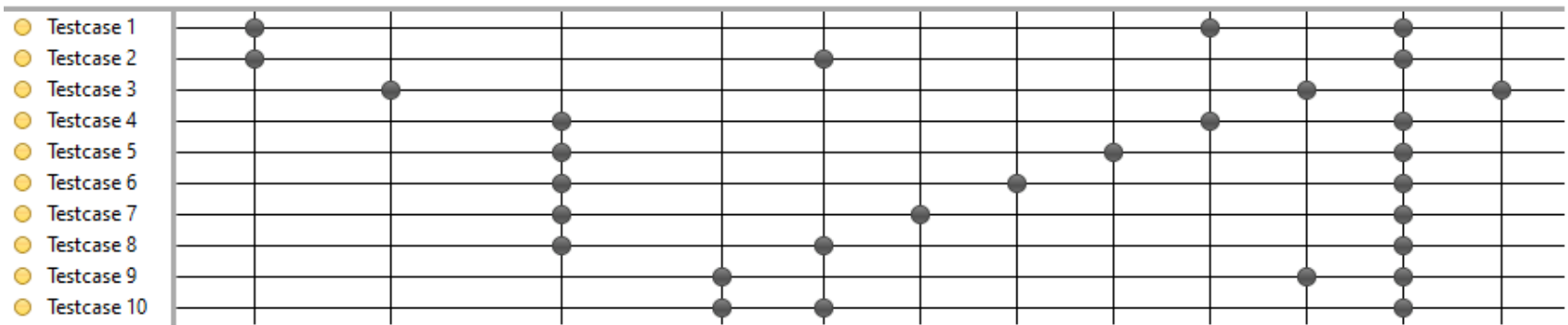
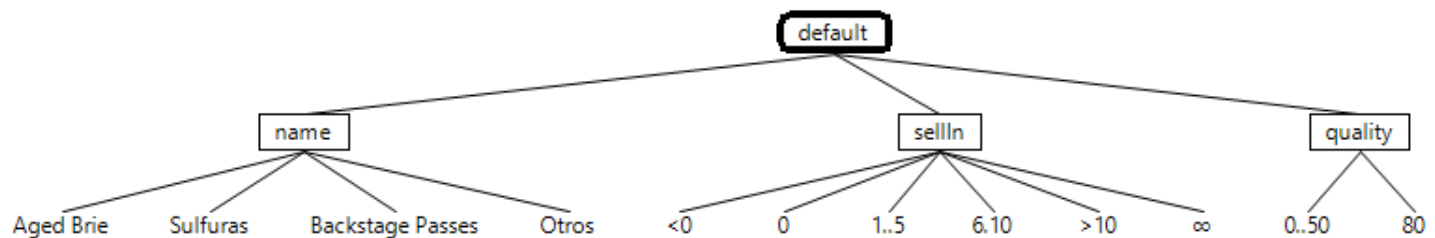
Casos de prueba

Prueba	name	sellIn	quality	Clases	Resultado		
					Pares	sellIn	quality
P1	"Aged Brie"	10	25	1 5 12	1-5, 1-12, 5-12	9	26
P2	"Aged Brie"	-8	25	1 6 12	1-6, 6-12	-9	27
P3	"Sulfuras"	*	80	2 11 13	2-11, 2-13, 11-13	*	80
P4	"Backstage passes"	11	25	3 7 12	3-7, 3-12, 7-12	10	26
P5	"Backstage passes"	9	25	3 8 12	3-8, 8-12	8	27
P6	"Backstage passes"	4	25	3 9 12	3-9, 9-12	3	28
P7	"Backstage passes"	0	25	3 10 12	3-10, 10-12	-1	0
P8	"Backstage passes"	-8	0	3 6 12	3-6	-9	0
P9	"Dexterity Vest"	10	20	4 5 12	4-5, 4-12	9	19
P10	"Dexterity Vest"	-8	25	4 6 12	4-12	-9	23

* Integer.MAX_VALUE

Casos de prueba

Representación gráfica como árbol de clasificación



Pero ya hay código escrito...

¡Con complejidad
ciclomática = 19!

```
public void updateQuality() {
    for (int i = 0; i < items.length; i++) {
        if (items[i].getName() != "Aged Brie"
            && items[i].getName() != "Backstage passes to a TAFKAL80EX concert") {
            if (items[i].getQuality() > 0) {
                if (items[i].getName() != "Sulfuras, Hand of Ragnaros") {
                    items[i].setQuality(items[i].getQuality() - 1);
                }
            }
        } else {
            if (items[i].getQuality() < 50) {
                items[i].setQuality(items[i].getQuality() + 1);

                if (items[i].getName() == "Backstage passes to a TAFKAL80EX concert") {
                    if (items[i].getSellIn() < 11) {
                        if (items[i].getQuality() < 50) {
                            items[i].setQuality(items[i].getQuality() + 1);
                        }
                    }

                    if (items[i].getSellIn() < 6) {
                        if (items[i].getQuality() < 50) {
                            items[i].setQuality(items[i].getQuality() + 1);
                        }
                    }
                }
            }
        }
    }

    if (items[i].getName() != "Sulfuras, Hand of Ragnaros") {
        items[i].setSellIn(items[i].getSellIn() - 1);
    }

    if (items[i].getSellIn() < 0) {
        if (items[i].getName() != "Aged Brie") {
            items[i].setQuality(0);
        }
    }
}
```

Casos de prueba

- Vamos a reforzarlos aplicando también la técnica de valores límite
 - ❖ De los valores límite posible, vamos a añadir solo algunos: aquellos en los que estimemos que es más probable que puedan provocar errores.

Prueba	name	sellIn	quality	Clases		Resultado	
						Pares	sellIn quality
P1	"Aged Brie"	10	25	1 5	12	1-5, 1-12, 5-12	9 26
P2	"Aged Brie"	-8	25	1 6	12	1-6, 6-12	-9 27
P3	"Sulfuras"	*	80	2 11	13	2-11, 2-13, 11-13	* 80
P4	"Backstage passes"	11	25	3 7	12	3-7, 3-12, 7-12	10 26
P5	"Backstage passes"	9	25	3 8	12	3-8, 8-12	8 27
P6	"Backstage passes"	4	25	3 9	12	3-9, 9-12	3 28
P7	"Backstage passes"	0	25	3 10	12	3-10, 10-12	-1 0
P8	"Backstage passes"	-8	0	3 6	12	3-6	-9 0
P9	"Dexterity Vest"	10	20	4 5	12	4-5, 4-12	9 19
P10	"Dexterity Vest"	-8	25	4 6	12	4-12	-9 23

Prueba	name	sellIn	quality	Clases		Resultado	
						Pares	sellIn quality
P1	"Aged Brie"	10	25	1 5	12	1-5, 1-12, 5-12	9 26
P1b	"Aged Brie"	10	50	valor límite 12			9 50
P2	"Aged Brie"	-8	25	1 6	12	1-6, 6-12	-9 27
P3	"Sulfuras"	*	80	2 11	13	2-11, 2-13, 11-13	* 80
P4	"Backstage passes"	11	25	3 7	12	3-7, 3-12, 7-12	10 26
P5	"Backstage passes"	9	25	3 8	12	3-8, 8-12	8 27
P6	"Backstage passes"	4	25	3 9	12	3-9, 9-12	3 28
P7	"Backstage passes"	0	25	3 10	12	3-10, 10-12	-1 0
P8	"Backstage passes"	-8	0	3 6	12	3-6	-9 0
P9	"Dexterity Vest"	10	20	4 5	12	4-5, 4-12	9 19
P10	"Dexterity Vest"	-8	25	4 6	12	4-12	-9 23

Prueba	name	sellIn	quality	Clases		Resultado	
						Pares	sellIn quality
P1	"Aged Brie"	10	25	1 5	12	1-5, 1-12, 5-12	9 26
P1b	"Aged Brie"	10	50	valor límite 12			9 50
P2	"Aged Brie"	-8	25	1 6	12	1-6, 6-12	-9 27
P2b	"Aged Brie"	-8	50	valor límite 12			-9 50
P2c	"Aged Brie"	-8	49	valor límite 12			-9 50
P2d	"Aged Brie"	0	25	valor límite 6			-1 27
P3	"Sulfuras"	*	80	2 11	13	2-11, 2-13, 11-13	* 80
P4	"Backstage passes"	11	25	3 7	12	3-7, 3-12, 7-12	10 26
P5	"Backstage passes"	9	25	3 8	12	3-8, 8-12	8 27
P6	"Backstage passes"	4	25	3 9	12	3-9, 9-12	3 28
P7	"Backstage passes"	0	25	3 10	12	3-10, 10-12	-1 0
P8	"Backstage passes"	-8	0	3 6	12	3-6	-9 0
P9	"Dexterity Vest"	10	20	4 5	12	4-5, 4-12	9 19
P10	"Dexterity Vest"	-8	25	4 6	12	4-12	-9 23

Prueba	name	sellIn	quality	Clases		Resultado	
						Pares	sellIn quality
P1	"Aged Brie"	10	25	1 5	12	1-5, 1-12, 5-12	9 26
P1b	"Aged Brie"	10	50	valor límite 12			9 50
P2	"Aged Brie"	-8	25	1 6	12	1-6, 6-12	-9 27
P2b	"Aged Brie"	-8	50	valor límite 12			-9 50
P2c	"Aged Brie"	-8	49	valor límite 12			-9 50
P2d	"Aged Brie"	0	25	valor límite 6			-1 27
P3	"Sulfuras"	*	80	2 11	13	2-11, 2-13, 11-13	* 80
P4	"Backstage passes"	11	25	3 7	12	3-7, 3-12, 7-12	10 26
P4b	"Backstage passes"	11	50	valores límite 7, 12			10 50
P5	"Backstage passes"	9	25	3 8	12	3-8, 8-12	8 27
P6	"Backstage passes"	4	25	3 9	12	3-9, 9-12	3 28
P7	"Backstage passes"	0	25	3 10	12	3-10, 10-12	-1 0
P8	"Backstage passes"	-8	0	3 6	12	3-6	-9 0
P9	"Dexterity Vest"	10	20	4 5	12	4-5, 4-12	9 19
P10	"Dexterity Vest"	-8	25	4 6	12	4-12	-9 23

Prueba	name	sellIn	quality	Clases		Resultado	
						Pares	sellIn quality
P1	"Aged Brie"	10	25	1 5 12		1-5, 1-12, 5-12	9 26
P1b	"Aged Brie"	10	50	valor límite 12			9 50
P2	"Aged Brie"	-8	25	1 6 12		1-6, 6-12	-9 27
P2b	"Aged Brie"	-8	50	valor límite 12			-9 50
P2c	"Aged Brie"	-8	49	valor límite 12			-9 50
P2d	"Aged Brie"	0	25	valor límite 6			-1 27
P3	"Sulfuras"	*	80	2 11 13		2-11, 2-13, 11-13	* 80
P4	"Backstage passes"	11	25	3 7 12		3-7, 3-12, 7-12	10 26
P4b	"Backstage passes"	11	50	valores límite 7, 12			10 50
P5	"Backstage passes"	9	25	3 8 12		3-8, 8-12	8 27
P5b	"Backstage passes"	10	50	valores límite 8, 12			9 50
P5c	"Backstage passes"	10	49	valores límite 8, 12			9 50
P5d	"Backstage passes"	6	25	valor límite 8			5 27
P5e	"Backstage passes"	6	50	valores límite 8, 12			5 50
P5f	"Backstage passes"	6	49	valores límite 8, 12			5 50
P6	"Backstage passes"	4	25	3 9 12		3-9, 9-12	3 28
P6b	"Backstage passes"	5	50	valores límite 9, 12			4 50
P6c	"Backstage passes"	5	49	valores límite 9, 12			4 50
P6d	"Backstage passes"	5	48	valores límite 9, 12			4 50
P6e	"Backstage passes"	1	25	valores límite 9			0 28
P6f	"Backstage passes"	1	50	valores límite 9, 12			0 50
P6g	"Backstage passes"	1	49	valores límite 9, 12			0 50
P6h	"Backstage passes"	1	48	valores límite 9, 12			0 50
P7	"Backstage passes"	0	25	3 10 12		3-10, 10-12	-1 0
P8	"Backstage passes"	-8	0	3 6 12		3-6	-9 0
P9	"Dexterity Vest"	10	20	4 5 12		4-5, 4-12	9 19
P10	"Dexterity Vest"	-8	25	4 6 12		4-12	-9 23

Prueba	name	sellIn	quality	Clases		Resultado	
						Pares	sellIn quality
P1	"Aged Brie"	10	25	1 5	12	1-5, 1-12, 5-12	9 26
P1b	"Aged Brie"	10	50	valor límite 12			9 50
P2	"Aged Brie"	-8	25	1 6	12	1-6, 6-12	-9 27
P2b	"Aged Brie"	-8	50	valor límite 12			-9 50
P2c	"Aged Brie"	-8	49	valor límite 12			-9 50
P2d	"Aged Brie"	0	25	valor límite 6			-1 27
P3	"Sulfuras"	*	80	2 11	13	2-11, 2-13, 11-13	* 80
P4	"Backstage passes"	11	25	3 7	12	3-7, 3-12, 7-12	10 26
P4b	"Backstage passes"	11	50	valores límite 7, 12			10 50
P5	"Backstage passes"	9	25	3 8	12	3-8, 8-12	8 27
P5b	"Backstage passes"	10	50	valores límite 8, 12			9 50
P5c	"Backstage passes"	10	49	valores límite 8, 12			9 50
P5d	"Backstage passes"	6	25	valor límite 8			5 27
P5e	"Backstage passes"	6	50	valores límite 8, 12			5 50
P5f	"Backstage passes"	6	49	valores límite 8, 12			5 50
P6	"Backstage passes"	4	25	3 9	12	3-9, 9-12	3 28
P6b	"Backstage passes"	5	50	valores límite 9, 12			4 50
P6c	"Backstage passes"	5	49	valores límite 9, 12			4 50
P6d	"Backstage passes"	5	48	valores límite 9, 12			4 50
P6e	"Backstage passes"	1	25	valores límite 9			0 28
P6f	"Backstage passes"	1	50	valores límite 9, 12			0 50
P6g	"Backstage passes"	1	49	valores límite 9, 12			0 50
P6h	"Backstage passes"	1	48	valores límite 9, 12			0 50
P7	"Backstage passes"	0	25	3 10	12	3-10, 10-12	-1 0
P8	"Backstage passes"	-8	0	3 6	12	3-6	-9 0
P9	"Dexterity Vest"	10	20	4 5	12	4-5, 4-12	9 19
P9b	"Dexterity Vest"	10	0	valor límite 12			9 0
P10	"Dexterity Vest"	-8	25	4 6	12	4-12	-9 23

Prueba	name	sellIn	quality	Clases		Resultado	
						Pares	sellIn quality
P1	"Aged Brie"	10	25	1 5 12		1-5, 1-12, 5-12	9 26
P1b	"Aged Brie"	10	50	valor límite 12			9 50
P2	"Aged Brie"	-8	25	1 6 12		1-6, 6-12	-9 27
P2b	"Aged Brie"	-8	50	valor límite 12			-9 50
P2c	"Aged Brie"	-8	49	valor límite 12			-9 50
P2d	"Aged Brie"	0	25	valor límite 6			-1 27
P3	"Sulfuras"	*	80	2 11 13		2-11, 2-13, 11-13	* 80
P4	"Backstage passes"	11	25	3 7 12		3-7, 3-12, 7-12	10 26
P4b	"Backstage passes"	11	50	valores límite 7, 12			10 50
P5	"Backstage passes"	9	25	3 8 12		3-8, 8-12	8 27
P5b	"Backstage passes"	10	50	valores límite 8, 12			9 50
P5c	"Backstage passes"	10	49	valores límite 8, 12			9 50
P5d	"Backstage passes"	6	25	valor límite 8			5 27
P5e	"Backstage passes"	6	50	valores límite 8, 12			5 50
P5f	"Backstage passes"	6	49	valores límite 8, 12			5 50
P6	"Backstage passes"	4	25	3 9 12		3-9, 9-12	3 28
P6b	"Backstage passes"	5	50	valores límite 9, 12			4 50
P6c	"Backstage passes"	5	49	valores límite 9, 12			4 50
P6d	"Backstage passes"	5	48	valores límite 9, 12			4 50
P6e	"Backstage passes"	1	25	valores límite 9			0 28
P6f	"Backstage passes"	1	50	valores límite 9, 12			0 50
P6g	"Backstage passes"	1	49	valores límite 9, 12			0 50
P6h	"Backstage passes"	1	48	valores límite 9, 12			0 50
P7	"Backstage passes"	0	25	3 10 12		3-10, 10-12	-1 0
P8	"Backstage passes"	-8	0	3 6 12		3-6	-9 0
P9	"Dexterity Vest"	10	20	4 5 12		4-5, 4-12	9 19
P9b	"Dexterity Vest"	10	0	valor límite 12			9 0
P10	"Dexterity Vest"	-8	25	4 6 12		4-12	-9 23
P10b	"Dexterity Vest"	-8	1	valor límite 12			-9 0
P10c	"Dexterity Vest"	-8	0	valor límite 12			-9 0
P10d	"Dexterity Vest"	0	25	valor límite 6			-1 23